Phytium飞腾

飞腾嵌入式腾珑 E2000
可信系统部署手册

（V1.1）

飞腾信息技术有限公司
www.phytium.com.cn
2023 年 05 月 12 日

版权所有 © 飞腾信息技术有限公司 2023。 保留一切权利。

未经本公司同意，任何单位、公司或个人不得擅自复制、翻译、摘抄本文档内容的部分或全部，不得以任何方式或途径进行传播和宣传。

商标声明

Phytium 和其他飞腾商标均为飞腾信息技术有限公司的商标。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

特别提示

本文档仅作为使用指导，飞腾对本文档内容不做任何明示或暗示的声明或保证。本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

由于产品版本升级或其他原因，本文档内容会不定期进行更新，如有变更，恕不另行通知。

最新技术资源

飞腾嵌入式软件开源社区： https://gitee.com/phytium_embedded

或者访问飞腾软件开发者平台： https://service.phytium.com.cn/developer/

目录

1 环境准备 1
1.1 硬件环境 1
1.2 软件环境 2
2 部署系统环境 3
2.1 概述 3
2.2 软件编译 3
2.2.1 系统要求 3
2.2.2 下载 buildroot 3
2.2.3 整体编译 3
2.3 固件打包 4
2.4 固件烧写 5
2.5 系统安装 6
2.6 系统启动 6
3 软件升级 8
3.1 概述 8
3.2 升级 PBF 8
3.3 升级 uboot 8
3.4 升级可信系统 9
3.4.1 系统编译 9
3.4.2 系统升级 9
3.5 升级 Linux 内核 9
3.5.1 系统编译 9
3.5.2 系统升级 10
4 调试 11
4.1 使用 T-boot 的方式调试 11
4.1.1 简介 11
4.1.2 原理 11
4.1.3 调试步骤 12

4.2 使用多启动方式更新固件 13
4.2.1 简介 13
4.2.2 原理 13
4.2.3 调试步骤 13
5 常见问题 16
5.1 Linux tee 设备文件不存在的问题 16
5.2 REE 侧程序运行时出现 GLIBC 版本不匹配的问题 17
5.3 TEE 固件不存在导致系统无法启动的问题 17
5.4 REE 侧 tee-supplicant 无法打印调试 log 的问题 17
5.5 多版本 openssl 导致固件打包工具错误的问题 17
A 术语和缩略语 19
B 更新记录 20

1 环境准备

1.1 硬件环境

本文档涉及到的主要硬件设备有以下几个：

1) 腾珑 E2000 参考板一块。

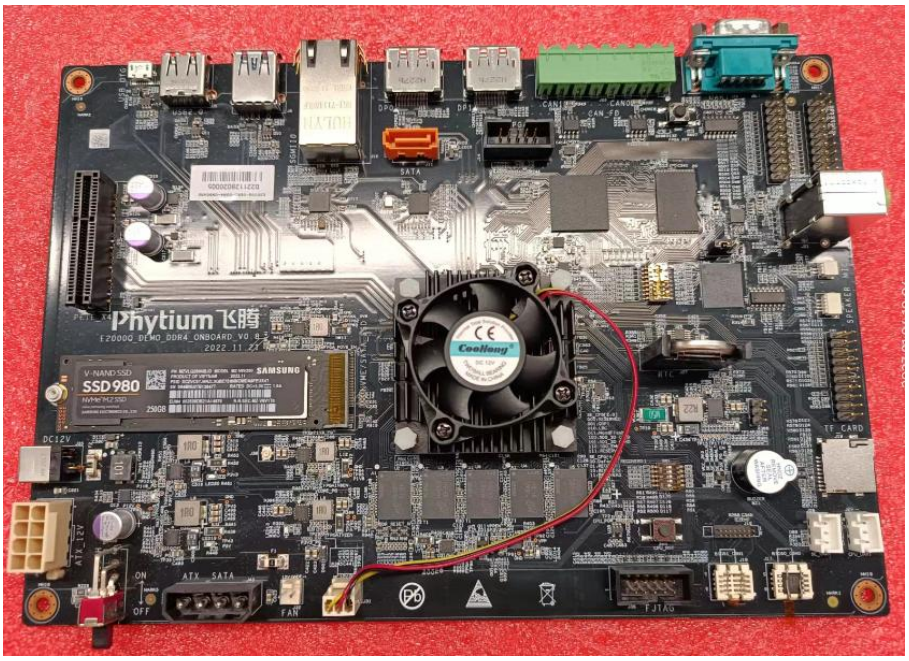

图 1.1 腾珑 E2000Q 参考板实拍图

2) 编译主机一台，可以是 ARM 主机或 x86 主机。

3) Flash 烧写器一台。

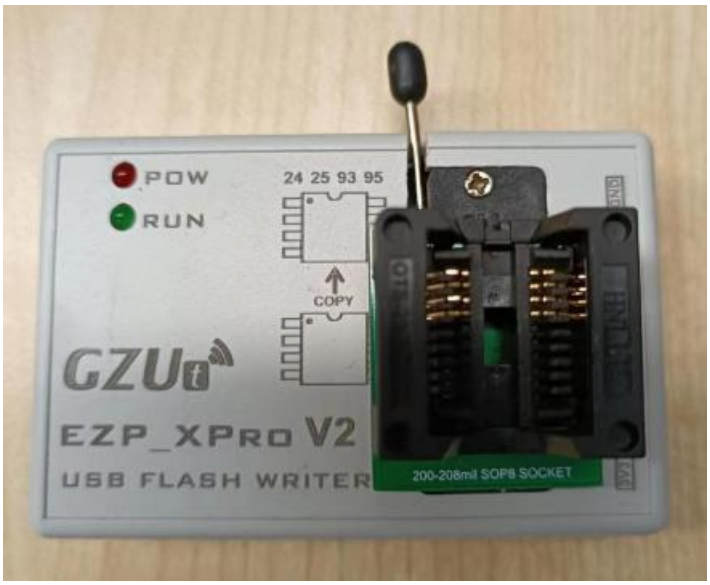

图 1.2 Flash 烧写器参考型号实拍图

1.2 软件环境

软件环境安装在编译主机上，推荐 x86 主机，并采用双系统方式。

Windows 系统用于安装 Flash 烧写器驱动，烧写 bios 芯片，调试打印输出显示，如果是 ARM 主机不方便安装 Windows 系统，可以考虑增加一台机器专门烧写 bios 芯片。

Linux 系统用于编译代码，生成映像文件，也可以作为打印输出显示，推荐使用 Ubuntu20.04 及以上系统，本文描述命令基于 Ubuntu 系统。

2 部署系统环境

2.1 概述

本章主要描述初次部署系统环境，推荐的操作顺序如下：

1) 使用 buildroot 的方式编译所有软件，相关软件都开源在 Gitee.com 上。
2) 使用 image_fix 工具，将所有固件打包成一个能整体烧写的 bin 文件。
3) 将 bin 文件烧写到 bios 芯片中。
4) 安装系统到 SATA 硬盘或 U 盘中。
5) 启动系统到应用环境中，执行 hello world 程序确认可信系统部署成功。

2.2 软件编译

关于 buildroot 的更多信息请参考飞腾嵌入式软件部在 Gitee.com 上开源的 phytium-linux-buildroot 仓库中的 README.md 文件。

2.2.1 系统要求

基于 buildroot 的编译大大简化了软件的编译过程，本文以 ubuntu20.04 系统上运行飞腾嵌入式软件部开源的 phytium-linux-buildroot 为例进行编译说明。

编译需要安装如下软件包：

```
$ sudo apt-get install debootstrap qemu-system-common qemu-user-static
binfmt-support
```

2.2.2 下载 buildroot

需要将飞腾开源的 phytium-linux-buildroot 仓库克隆下载到本地。

```
$ git clone git@gitee.com:phytium_embedded/phytium-linux-buildroot.git
```

2.2.3 整体编译

1) 配置 defconfig

使用 phytium_e2000_ubuntu_defconfig 作为基础配置项，并合并支持 TEE 的配置：

```
$ cd phytium-linux-buildroot
$ ./support/kconfig/merge_config.sh configs/phytium_e2000_ubuntu_defconfig
configs/phytium_e2000_optee.config
```

注意：需要 phytium_e2000_defconfig 就将传给 merge_config.sh 的第一个参数改为 configs/phytium_e2000_defconfig，其他 defconfig 类似。

2) 编译

```
$ make
```

3) 镜像的输出位置

镜像的输出主目录是 output/images。

根文件系统是 output/images/rootfs.tar。

tee-supplican，xtest 等执行文件放在根文件目录下/usr/bin。

libteec.so 等动态链接库放在根文件目录下/usr/lib。

TA 放在根文件目录下/data/optee_armtz。

TEE OS 是 output/images/tee*.bin，其中‘*’根据配置会有不同表示，同时，该 bin 文件也要作为下一步固件打包的链接目标文件。

CA & TA 需要的头文件和库已被安装的 buildroot 的 STAGING_DIR 内，位置是 output/host/aarch64-buildroot-linux-gnu/sysroot。

注意：后续如果需要在 bootroot 包中添加 CA & TA，编译 CA 则不需要额外指定头文件目录，而编译 TA 请设置变量 TA_DEV_KIT_DIR=$(PHYTIUM_OPTEE_OS_SDK)，PHYTIUM_OPTEE_OS_SDK 环境变量的实际路径为$(STAGING_DIR)/lib/optee/export-ta_arm64。

2.3 固件打包

编译完成后，输出目录中的 TEE OS 镜像作为一种固件，需要和其他固件打包在一起，并重新烧写 BIOS 芯片。

固件打包工具为飞腾自研工具 image_fix_test_tee.tar，是一个未压缩的 tar 包。里面包含了一套可用的固件 image，包括飞腾自研基础固件 PBF 和飞腾基于通用 uboot 生成的通用固件。该固件打包工具可以联系飞腾支持人员获取，暂不发布在开源社区。

固件打包工具中有两个软链接，分别是 bl32_new.bin 和 bl33_new.bin，请先将 bl32_new.bin 指向 TEE OS 镜像，例如可指向 output/images/tee*.bin，然后在打包工具根目录下执行以下命令进行整体打包：

```
./my_scripts/image-fix.sh bl32
```

注意：image_fix_test_tee.tar 中有 readme 文件提示打包工具的一些操作和配置，请仔细阅读。打包完成后通过烧片器将打包后的 bin 文件烧写到 BIOS 芯片中即可。

2.4 固件烧写

客户需要准备 Flash 烧写器及 SOP8 底座各一个，按要求在 PC 机上安装烧录器驱动，下图以 EZP_XPro 编程器烧写过程示例说明。

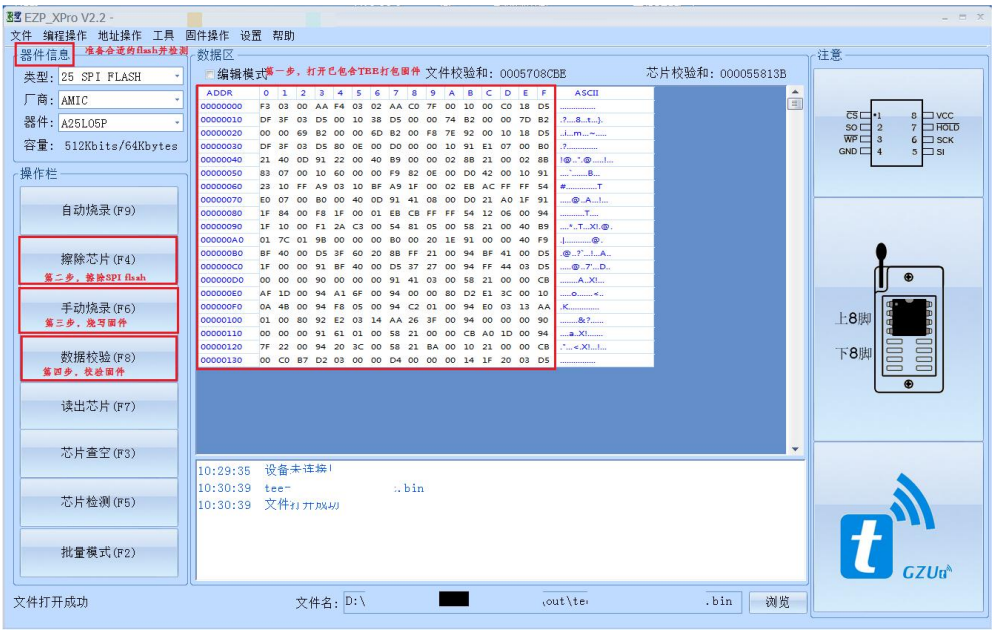

图 2.1 EZP_XPro 编程器烧写固件说明图

从 EZP_XPro 编程器中取出已经烧写成功的 Flash 芯片，并安装在 E2000 板上，下图以 E2000D 参考板示例说明芯片安装位置。

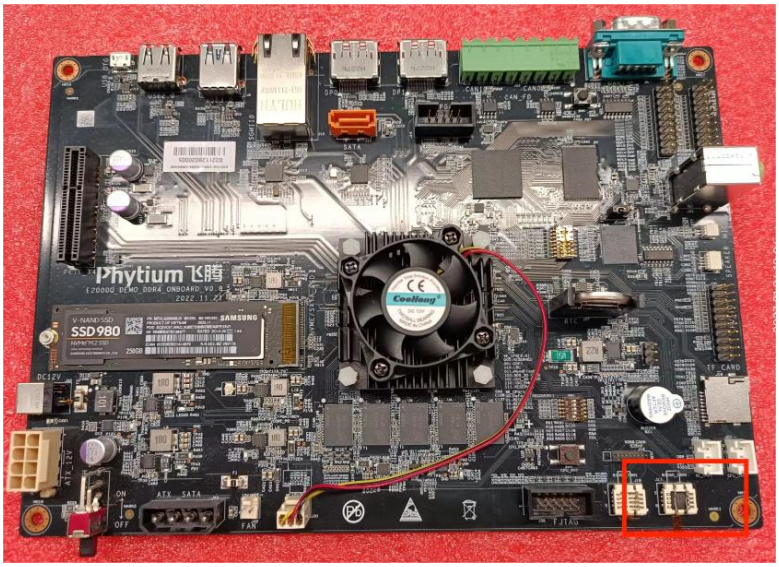

图 2.2 固件芯片在 E2000D 参考板实拍图上的位置

2.5 系统安装

1) 主机端将目标盘，比如 SATA 盘或 U 盘分成两个分区。

注意：本文以主机识别的设备名为/dev/sdb 为例进行说明，请按实际识别设备名进行相应更改。

```
$ sudo fdisk /dev/sdb
```

2) 主机端将内核和设备树拷贝到第一个分区，将根文件系统拷贝到第二个分区。

```
$ sudo mkfs.ext4 /dev/sdb1
$ sudo mkfs.ext4 /dev/sdb2
$ sudo mount /dev/sdb1 /mnt
$ sudo cp phytium-linux-buildroot/output/images/Image /mnt
$ sudo cp phytium-linux-buildroot/output/images/e2000*.dtb /mnt
$ sudo cp phytium-linux-buildroot/output/images/tee*.bin /mnt
$ sync
$ sudo umount /dev/sdb1
$ sudo mount /dev/sdb2 /mnt
$ sudo cp phytium-linux-buildroot/output/images/rootfs.tar /mnt
$ cd /mnt
$ sudo tar xvf rootfs.tar
$ sync
$ cd ~
$ sudo umount /dev/sdb2
```

2.6 系统启动

SATA 盘或 U 盘接到开发板，启动开发板电源，串口输出 U-Boot 命令行，设置 U-Boot 环境变量，启动文件系统。

SATA 盘运行命令参考如下：

```
setenv bootargs console=ttyAMA1,115200 audit=0 earlycon=pl011,0x2800d000
root=/dev/sda2 rw
ext4load scsi 0:1 0x90100000 Image
ext4load scsi 0:1 0x90000000 e2000q-demo-board.dtb
booti 0x90100000 - 0x90000000
```

U 盘运行命令参考如下：

```
setenv bootargs console=ttyAMA1,115200    audit=0 earlycon=pl011,0x2800d000
root=/dev/sda2 rootdelay=5 rw;
usb start
ext4load usb 0:1 0x90100000 Image;
ext4load usb 0:1 0x90000000 e2000q-demo-ddr4.dtb;
```

```
booti 0x90100000 - 0x90000000
```

Ubuntu 启动成功后可以看到：

```
[  OK  ] Started phytium tee supplicant.
```

执行 hello_world：

```
# hello_world
Invoking TA to increment 42
TA incremented value to 43
```

这个 hello world 演示程序实际是 REE 环境下发了一个值为 42 的变量给了 TEE 环境，TEE 环境收到后，进行了自增操作，然后返回给 REE 环境。具体可以参考源代码 hello_world.c（REE 环境运行）和 hello_world_ta.c（TEE 环境运行）。

3 软件升级

3.1 概述

系统部署完毕之后，如果需要针对环境中的主要软件修改升级更新，或替换不同版本。本章主要描述怎么单独升级环境中的某一个软件。

3.2 升级 PBF

PBF 作为飞腾自研的基础固件，其作用等同于 ATF 启动流程中的 BL1、BL2 和 BL31。一般情况下是不需要升级的。

飞腾在对基础固件有修改时，会发布新的 PBF 的升级包，升级包中包括一个 bin 文件和升级说明。将升级包中的 bin 文件，替换 image_fix 打包工具 build 目录下的 bin 文件，然后重新打包、烧写 bios 芯片，即完成 PBF 的升级。

3.3 升级 uboot

该版本固件默认配置的 bl33 的启动偏移地址为 bl33_base 参数定义，如需修改，请打开文件./my_scripts/image-fix.sh，修改文件开头的 bl33_base 参数即可，同时，修改了该参数后，bl33 镜像内部也需要将引导地址替换为该地址。

如果需要使用当前版本的 uboot 镜像，可以从固件打包工具中提取使用，当前版本的 uboot 镜像文件是 bl33_new.bin 软链接指向的文件。

如果 uboot 镜像需要单独升级更新，可以在固件打包工具中将软链接 bl33_new.bin 指向待更新的 uboot 镜像，然后执行以下命令重新打包：

```
my_scripts/image-fix.sh bl33
```

注意固件打包工具中有 readme 文件提示打包工具的一些操作和配置，请仔细阅读。

打包完成后通过 Flash 烧写器将打包后的 bin 文件烧写到 BIOS 芯片中即完成 uboot 的更新升级。

3.4 升级可信系统

3.4.1 系统编译

1) 获取 OPTEE 内核源码

```
git clone git@gitee.com:phytium_embedded/Phytium-optee.git
```

2) 配置交叉编译环境

建议使用和 buildroot 编译是相同的 toolchain，具体位置在：

phytium-linux-buildroot/dl/toolchain-external-arm-aarch64/

```
export PATH=$PATH:[your toolchain path]/bin ARCH=arm64
CROSS_COMPILE64=[cross compiler prefix]
cd Phytium-optee
./build_all e2000ddemo d
```

编译脚本 build_all 直接运行会有 help 打印，请参考其中的参数说明。

编译没有错误 log 打印后，编译出来的 OPTEE OS 镜像存放在编译目录中新增的 out 目录下。

3.4.2 系统升级

请参考本文档 2.3 固件打包、2.4 固件烧写进行 TEE OS 内核镜像的升级。

3.5 升级 Linux 内核

3.5.1 系统编译

1) 获取内核源码

```
git clone git@gitee.com:phytium_embedded/phytium-linux-kernel.git
```

buildroot 默认使用的是 linux-5.10 分支的代码。

2) 配置交叉编译环境

此处建议使用和 buildroot 相同的 toolchain。buildroot 的 toolchain 放在：

phytium-linux-buildroot/dl/toolchain-external-arm-aarch64/

```
export PATH=$PATH:[your toolchain path]/bin ARCH=arm64
CROSS_COMPILE=[cross compiler prefix]
cd phytium-linux-kernel
make e2000_defconfig e2000_optee.config
make
make modules_install INSTALL_MOD_PATH=[module path]
```

3.5.2 系统升级

将编译生成的内核和设备树文件，具体路径如下：

phytium-linux-kernel/arch/arm64/boot/Image

phytium-linux-kernel/arch/arm64/boot/dts/phytium/*.dtb

复制到系统盘的第一个分区，具体操作方法，请参考本文档 2.5 系统安装。

外部模块：

[module path]/lib/* 复制到板子根文件系统中/lib。

4 调试

4.1 使用 T-boot 的方式调试

4.1.1 简介

由于可信系统的内核是作为固件和其他固件一起保存在 bios 芯片中的，这就会导致调试可信系统内核时，需要不停重复打包、烧片的繁重工作。

T-boot 是一种方便调试可信系统的方法，由飞腾自研完成，主要目的是避免在调试可信系统过程中频繁打包、烧片的问题。

4.1.2 原理

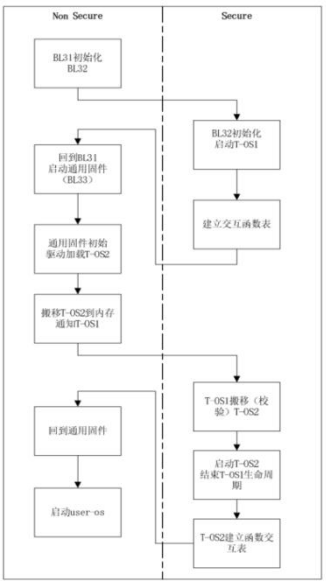

图 4.1 飞腾公司画的 T-boot 交互原理图

如上图所示，流程的改变从 BL31 启动 BL32 开始，BL31 和原本的 BL31 并不一样，支持 T-boot 的 BL31 会将 bios 芯片中打包的 BL32（即图中的 T-OS1）加载到一个临时位置，然后启动 BL33（即图中的通用固件，本文对应的是 uboot）。

然后，在 uboot 环境中，通过命令将真正的 BL32（即图中的 T-OS2）加载到内存，并通知 T-OS1。T-OS1 搬移 T-OS2 到真正的内存位置，并启动 T-OS2，完成 T-OS1 的使命。注意：uboot 并不能直接把 T-OS2 加载到正确的内存，是因为 uboot 是 Non Secure 世界的软件，无法访问 Secure 世界的内

存。

最后，真正的可信系统（T-OS2）启动，完成初始化，并回到正常的启动流程中。

通过这种方式，可以让真正的可信系统内核，通过 U 盘或者 SD 卡等多种途径，加载到启动流程中来，避免了一定需要烧写到 bios 芯片中的问题。

不过，这种方式和正常的方式相比，存在安全性问题，不建议作为正常产品，只建议作为一种调试手段。

4.1.3 调试步骤

调试开始前，先检查下以下准备工作是否完成：

a) 支持 T-boot 的固件已经烧录到了 BIOS 芯片中。

b) 待调试的 TEE OS 映像已经准备好，默认名字叫 tee-phytium-e2000demo.bin。

c) REE 侧的支持包 data-e2000demo-debug.tgz 已经导入到启动文件系统中，或者导入到文件系统可以访问到的存储介质中。

d) 基础运行环境就绪。即：在没有 TEE OS 的情况下，系统能正常启动到 Linux Shell 界面中。包括相关 Linux 映像都已经准备好，设备树设置好。

准备好后，推荐按以下调试步骤进行调试：

1) 启动系统到 uboot 命令行界面

2) 启动 USB 支持 usb start

如果没有任何反应，可以考虑先执行 usb stop，再重新执行 usb start。同理，如果这时候要插拔 U 盘，也可以 usb stop 下，然后插拔 U 盘，再 usb start。

3) 将 TEE OS 映像加载到内存中临时存放

```
fatload usb 0:1 0x90100000 tee-phytium-e2000demo.bin 200000
```

注意：TEE OS 映像大小通常不会超过 2MB 大小，该大小不影响 linux 系统加载启动，默认写 20000 大小，也可以写映像的实际大小，固件打包时默认按 2MB 预留大小

可以通过 usb strorge 命令查看下有没有 storage 设备，确定 U 盘的设备号和分区号。usb strorge 命令没有显示分区号，但可以通过查询 U 盘内容来确定分区号。

一般来说，U 盘只分一个区，分区号就是 1。可以尝试用 ls usb 读 U 盘内容的方式扫一下分区号看看。最后一个参数是大小，文件明确的时候可以不指定大小。

4) 调用 bl31 服务，完成 TEE OS 的替换。

```
tboot 90100000 200000
```

4.2 使用多启动方式更新固件

4.2.1 简介

对于腾珑 E2000 参考板来说，默认的启动方式是通过 QSPI Flash 的方式启动，即如本文档 2.4 节描述的方式，先通过 Flash 烧写器烧写固件到 Flash 芯片中，然后再讲芯片放到对应芯片插槽中，最后上电启动。这种方式在调试过程中，有可能导致 Flash 芯片反复插拔、烧写，比较麻烦，本节主要说明下怎么通过腾珑 E2000 丰富的多启动方式来比较方便地更新固件，让开发者摆脱烧写器的束缚。

4.2.2 原理

腾珑 E2000 参考板支持多种启动方式，那这样，可以通过其他启动方式，先进入一个类似于手机 recovery 的环境，在这个环境中去刷新目标启动方式的固件，最后再切回目标启动方式即可。这样，即便刷固件出现问题，也可以重新再刷，不会刷成砖头。

参考板多种启动方式，如下图所示：

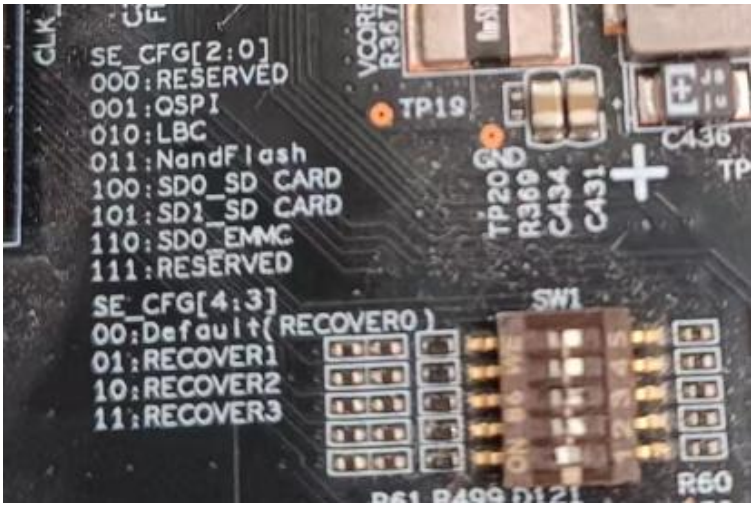

图 4.2 多启动方式示意实拍图

本文档主要考虑其中的 QSPI（001）、SD1_SD CARD（101）和 SD0_EMMC（110）三种启动方式，并进行了测试。图中目前拨码开关 SW1 的状态是 001，拨码开关上有 bit 位的标识，ON 表示 1。

4.2.3 调试步骤

以 SD 卡启动作为 recovery 状态，QSPI 启动作为正常启动状态为例进行说明，其他组合方式操作是差不多的。具体步骤如下：

1) 将需要更新的固件文件 fip-all.bin 拷贝到 U 盘中，SD 卡插入参考板。

2) SW1 拨到 001，从 QSPI（老固件）启动进入 uboot 命令行环境。

3) 选中需要操作的设备，命令和 log 如下：

```
E2000#mmc dev 1
switch to partitions #0, OK
mmc1 is current device
```

这里是选中 SD1_SD CARD，如果要选中 SD0_EMMC，参数由‘1’修改为‘0’。

4) 将 QSPI 中的老固件拷贝到内存中。

因为老固件可以正常启动，可以作为 recovery 环境的固件，先把老固件移动到 SD 卡中，作为 SD 卡启动方式用的固件。命令如下：

```
E2000#cp.b 0 90000000 300000
```

使用 cp.b 命令时，0 表示 QSPI Flash 的首地址，90000000 可以作为临时内存 buffer，300000 表示拷贝大小，一般固件不超过 3MB。注意：uboot 命令参数默认是 16 进制数。

5) 将内存中老固件刷到 SD 卡的首地址空间。

完成的命令和 log 如下：

```
E2000#mmc write 90000000 0 1800
MMC write: dev # 1, block # 0, count 6144 ... 6144 blocks written: OK
```

其中，0x1800 这个参数表示写入 blocks 数，因为 SD 卡必须按 block 读写，每个 block 大小是 512B，所以 0x1800 大概是略大于 3MB 左右。

6) SW1 拨到 101，从 SD 卡（老固件）启动进入 uboot 命令行环境。

7) 启动 U 盘支持。

命令和部分 log 如下：

```
E2000#usb start
starting USB...
Bus usb3@31a00000: Register 2000110 NbrPorts 2
Starting the controller
```

8) 将 U 盘中保存的新固件拷贝到内存中。

命令和 log 如下：

```
E2000#fatload usb 0:1 90000000 tmptest/fip-all.bin 400000
3269432 bytes read in 195 ms (16 MiB/s)
```

这里的参数中‘0:1’表示分区，需要根据 U 盘实际情况确定，90000000 依然是内存的临时 buffer 区域，PATH 参数要根据实际情况进行修改，400000 是读取文件的最大 size 值，可以输入大一点，实际操作的大小是以文件大小为准，可以从 log 看出该次实际操作的字节数是 3269432，log 是十进制数显示的，可以在打包环境中确认 fip-all.bin 文件的大小就是这个值。

9) 将新固件从内存中拷贝到 QSPI Flash 中。

命令和 log 如下：

```
E2000#flashe 0 400000
erase qspi flash
erase done
E2000#flashw 90000000 0 400000
write ...
write done
```

注意这里需要先 erase 再 write，这和 Flash 的硬件特性有关，400000 依然是一个最大值，大于新固件实际文件大小即可。其他参数和上面步骤定义保持一致。

10) SW1 拨到 001，重新启动，这时候就是新固件启动流程了。

由于可信系统固件属于启动固件的一种，所以，通过以上调试方式，能完成启动固件中可信系统固件的更新和调试，避免反复使用烧写器导致效率低下。

注意：

a) 如果没有 SD 卡，可以考虑使用 SD0_EMMC 作为 recovery 环境，区别在于步骤中的 mmc dev 后面修改为 0。

b) 如果没有 QSPI Flash，也可以通过普通主机环境中 dd 命令，将 fip-all.bin 拷贝入 SD 卡中启动分区，正常启动后再把 SD0_EMMC 作为 recovery 环境，命令类似于：

```
dd if=fip-all.bin of=/dev/sdb
```

c) 步骤中使用的 uboot 命令是飞腾 uboot 特有的，可以从飞腾技术支持获取相关源码。

5 常见问题

本章主要收录飞腾腾珑 E2000 可信系统部署和调试过程中遇到的一些常见问题，仅供参考。

5.1 Linux tee 设备文件不存在的问题

请按如下顺序进行确认：

1) 确认设备树中 optee 有 enable

```
cat /sys/firmware/devicetree/base/firmware/optee/status
```

没有 status 节点时默认是 enable 的。如果 status 节点是 disable，请改为 okay 并编译替换设备树。

2) 确认内核 CONFIG 有 enable OPTEE

```
zcat /proc/config.gz | grep OPTEE
```

如果 CFG_OPTEE is not set，请确认有按照如下方式编译：

a) 编译 linux kernel

```
make e2000_defconfig e2000_optee.config
make
```

b) 编译 buildroot：

```
./support/kconfig/merge_config.sh configs/phytium_e2000_ubuntu_defconfig \
                                  configs/phytium_e2000_optee.config
make
```

3) 确认 optee log 正常

```
dmesg | grep optee
```

正常情况的 log 类似如下：

```
optee: probing for conduit method.
optee: revision 3.12 (7743059c)
optee: dynamic shared memory is enabled
optee: initialized driver
```

如果是 tboot 方式但忘记加载 TEE OS，或者 TEE OS 镜像有问题，会是如下 log：

```
optee: probing for conduit method.
optee: api uid mismatch
optee: probe of firmware:optee failed with error -22
```

5.2 REE 侧程序运行时出现 GLIBC 版本不匹配的问题

以 Ubuntu20.04 上运行 tee-supplicant 时出现如下异常为例：

```
./tee-supplicant: /lib/aarch64-linux-gnu/libc.so.6: version `GLIBC_2.33' not found
(required by ./tee-supplicant)
```

这是因为 ubuntu20.04 自带的 glibc 版本是 2.31，而使用交叉编译工具链包含的 glibc 版本是 2.33。因此请保证编译工具链的 glibc 版本不高于文件系统中带的 glibc 版本。

5.3 TEE 固件不存在导致系统无法启动的问题

该问题是因为采用 buildroot 编译出来的系统，如果默认包含 TEE 固件，系统在启动的时候需要去查询 TEE 固件版本，如果 TEE 固件没有正确启动，会导致系统启动进入 TEE 侧后无法返回，导致系统无法正常启动。

可以通过 log 打印确认 TEE OS 是否已经正常启动。

5.4 REE 侧 tee-supplicant 无法打印调试 log 的问题

通过 buildroot 生成、运行在 E2000 板上的 ubuntu 系统已经将 tee-supplicant 放到 systemd 启动，log 不会直接打印到 stdout，请使用获取 systemd 获取 log 的方式，如：journalctl | grep tee-supplicant。

5.5 多版本 openssl 导致固件打包工具错误的问题

X86+Ubuntu 使用固件打包工具 image_fix_test_tee 时，如果安装了多个版本的 openssl 后，有可能出现以下错误：

```
libssl.so.1.0.0:no version information available
```

解决方法：

下载 openssl-1.0.2.tar，编译安装后，生成的 libcrypto.so.1.0.0，放入如/home/.../GmSSL-master 后，使用 export LD_LIBRARY_PATH=/home/.../GmSSL-master 指定目录。

A 术语和缩略语

名称	全称	解释
OPTEE	Open Portable TEE	一个开源的、可信执行环境运行软件实现的项目，包括操作系统、运行在非安全执行环境中的交互驱动、支持程序和库。
TEE	Trusted Execution Environment	可信的执行环境，在 ARM 的安全架构中，属于安全世界的执行环境。
REE	Rich Execution Environment	丰富的执行环境，在 ARM 的安全架构中，属于非安全世界的执行环境。
TA	Trusted Application	运行在安全世界的应用程序。
CA	Client Application	运行在非安全世界的应用程序，负责和安全世界的应用程序进行交互。

B 更新记录

更新日期	版本号	说明
2023-02-21	1.0	首次发布。
2023-05-12	1.1	更新模板、字体，增加多启动方式调试功能描述。